ONE8 FOOTWEAR E-COMMERCE DATABASE

DBMS Task: Advanced SQL Query Implementation

Objective: Implement subqueries and nested queries, find products above average price, identify customers with maximum purchases, write complex business queries and prepare an advanced SQL report.

Tables Used

Table	Purpose
Customers	Customer information and city.
Products	ONE8 footwear product, category and price.
Orders	Customer order date and order status.
Order_Details	Products, quantities and unit prices in each order.
Payments	Payment method, amount and status.

Concept: Subqueries and Nested Queries

A subquery is a SELECT statement written inside another SQL statement. The inner query executes to provide a value or result to the outer query. A nested query may contain more than one level of SELECT statements. A correlated subquery refers to a column from the outer query and is evaluated in relation to each outer row.

1. Find Products Above Average Price

```
SELECT Product_ID,Product_Name,Price
FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products)
ORDER BY Price DESC;
```

The inner query calculates the average price of all products. The outer query displays products whose price is greater than that average.

2. Products Above Their Category Average

```
SELECT p.Product_Name,p.Category,p.Price
FROM Products p
WHERE p.Price > (
 SELECT AVG(p2.Price)
 FROM Products p2
 WHERE p2.Category=p.Category
);
```

This correlated subquery compares each product's price with the average price of products in the same category.

3. Customers Spending Above the Average Customer Purchase

```
SELECT c.Customer_Name,
       SUM(d.Quantity*d.Unit_Price) AS Purchase_Total
FROM Customers c
JOIN Orders o ON c.Customer_ID=o.Customer_ID
JOIN Order_Details d ON o.Order_ID=d.Order_ID
GROUP BY c.Customer_ID,c.Customer_Name
HAVING SUM(d.Quantity*d.Unit_Price) > (
 SELECT AVG(Customer_Total)
 FROM (
  SELECT SUM(d2.Quantity*d2.Unit_Price) AS Customer_Total
  FROM Orders o2
  JOIN Order_Details d2 ON o2.Order_ID=d2.Order_ID
  GROUP BY o2.Customer_ID
 ) AS Customer_Averages
);
```

The nested query calculates each customer's total purchase and then finds the average of those totals. The outer query returns customers whose spending is above that average.

4. Identify Customer(s) with Maximum Purchases

```
SELECT c.Customer_Name,
       SUM(d.Quantity*d.Unit_Price) AS Maximum_Purchase
FROM Customers c
JOIN Orders o ON c.Customer_ID=o.Customer_ID
JOIN Order_Details d ON o.Order_ID=d.Order_ID
GROUP BY c.Customer_ID,c.Customer_Name
HAVING SUM(d.Quantity*d.Unit_Price) = (
 SELECT MAX(Customer_Total)
 FROM (
  SELECT SUM(d2.Quantity*d2.Unit_Price) AS Customer_Total
  FROM Orders o2
  JOIN Order_Details d2 ON o2.Order_ID=d2.Order_ID
```

```
   GROUP BY o2.Customer_ID
  ) AS Customer_Totals
 );
```

The inner query calculates purchase totals for each customer. MAX() finds the highest total, and the outer query returns the customer or customers who achieved it, including ties.

5. Complex Business Query: Products Never Purchased

```
SELECT p.Product_Name,p.Category,p.Price
FROM Products p
WHERE NOT EXISTS (
 SELECT 1 FROM Order_Details d
 WHERE d.Product_ID=p.Product_ID
);
```

NOT EXISTS checks whether no matching order-detail record exists for a product. This helps identify products that have not yet been purchased.

6. Nested Query: Customers Who Purchased Every Running Shoes Product

```
SELECT c.Customer_Name
FROM Customers c
WHERE NOT EXISTS (
 SELECT 1 FROM Products p
 WHERE p.Category='Running Shoes'
 AND NOT EXISTS (
  SELECT 1 FROM Orders o
  JOIN Order_Details d ON o.Order_ID=d.Order_ID
  WHERE o.Customer_ID=c.Customer_ID
  AND d.Product_ID=p.Product_ID
 )
);
```

The double NOT EXISTS pattern checks that there is no Running Shoes product missing from a customer's purchase history.

7. Advanced SQL Reports

The accompanying SQL file includes additional reports: products with sales quantity above the average quantity per product; customer-wise order and purchase totals; categories whose sales are above average category sales; and orders whose total value is greater than the average order value.

How to Run

1. Open MySQL Workbench and connect to your MySQL server.
2. Open ONE8_Advanced_SQL_Queries.sql.
3. Review the DROP TABLE commands. They recreate the sample tables and delete existing data in those tables.
4. Execute the script. Run each SELECT query separately if you want to view its result independently.

Conclusion

This advanced SQL task applies scalar, nested and correlated subqueries, aggregate functions, HAVING, and EXISTS/NOT EXISTS conditions. The queries analyse product prices, customer purchase behaviour, product demand, category performance and high-value orders for the ONE8 footwear e-commerce database.